

R: GROUP BY ... MUTATE ...

```
df |>
  group_by(artist) |>
  mutate(lowest.rank = max(rank))
```

A grouped_df: 50 × 8

date	rank	song	artist	last.week	peak.rank	weeks.on.board	top.rank
<chr>	<int>	<chr>	<chr>	<int>	<int>	<int>	<int>
1/30/2021	1	Drivers License	Olivia Rodrigo	1	1	2	1
1/30/2021	2	34+35	Ariana Grande	11	2	12	5
1/30/2021	3	Mood	24kGoldn Featuring iann dior	2	1	24	7
1/30/2021	4	Blinding Lights	The Weeknd	4	1	59	4
1/30/2021	5	Positions	Ariana Grande	3	1	13	5
1/30/2021	6	Levitating	Dua Lipa Featuring DaBaby	6	6	16	10

R: GROUP BY ... MUTATE ...

```
df |>
  group_by(artist) |>
  mutate(lowest.rank = max(rank)) |>
  arrange(artist)
```

date	rank	song	artist	last.week	peak.rank	weeks.on.board	top.rank
<chr>	<int>	<chr>	<chr>	<int>	<int>	<int>	<int>
1/30/2021	3	Mood	24kGoldn Featuring iann dior	2	1	24	7
1/23/2021	2	Mood	24kGoldn Featuring iann dior	1	1	23	7
1/16/2021	1	Mood	24kGoldn Featuring iann dior	1	1	22	7
1/9/2021	1	Mood	24kGoldn Featuring iann dior	7	1	21	7
1/2/2021	7	Mood	24kGoldn Featuring iann dior	5	1	20	7
1/30/2021	9	Bang!	AJR	8	8	29	9
1/23/2021	8	Bang!	AJR	9	8	28	9
1/16/2021	9	Bang!	AJR	11	9	27	9
1/2/2021	5	It's The Most Wonderful Time Of The Year	Andy Williams	7	5	20	5
1/30/2021	2	34+35	Ariana Grande	11	2	12	5
1/30/2021	5	Positions	Ariana Grande	3	1	13	5

R: GROUP BY ... SUMMARIZE ...

```
df |>
  group_by(artist) |>
  summarize(lowest.rank = max(rank))
```

A tibble: 23 × 2

artist	top.rank
<chr>	<int>
24kGoldn Featuring iann dior	7
AJR	9
Andy Williams	5
Ariana Grande	5
BTS	5
Bobby Helms	3

R: GROUP BY ... SUMMARIZE ...

```
df |>  
  group_by(artist) |>  
  summarize(count = n())
```

A tibble: 23 × 2

artist	count
<chr>	<int>
24kGoldn Featuring iann dior	5
AJR	3
Andy Williams	1
Ariana Grande	5
BTS	1
Bobby Helms	1

Compatible aggregation functions: `max()`, `min()`,
`median()`, `n()`, `sd()`, `sum()`, `length()`

9 - SAS: READING IN FILES

IN THE TERMINAL

You invoke programs or commands by their name,
then space, then options:

IN THE TERMINAL

You invoke programs or commands by their name,
then space, then options:

- `ping google.com` tells ping program to ping the URL.

IN THE TERMINAL

You invoke programs or commands by their name,
then space, then options:

- `ping google.com` tells ping program to ping the URL.
- SAS works like this...

PUNCH CARD ERA



- Typically store 1 observations per card.
- 1 card has 80 columns (SAS defaults to this too.)
- Variables (fields) line up between cards lowers error: so space-delimited fields > CSV

GO TO SAS STUDIO

<https://welcome.oda.sas.com/>

The screenshot shows the SAS OnDemand for Academics welcome page. The browser address bar displays welcome.oda.sas.com. The page header includes "SAS® OnDemand for Academics" on the left, "United States 1" in the center, and a user profile "Man Tsui" on the right. The main content area features a "SAS® Studio" section with the text "Write and run SAS code with a Web-based SAS development environment." and a prominent blue "Launch" button. Below this, under "Other Actions:", there are links for "Learn" and "Clear my saved tabs". At the bottom, a "SAS Training" banner shows a person working on a laptop, with the text "Master analytics skills—start learning free with expert-led SAS training." and a "Learn More" link. The footer contains links for "License", "Terms of Use & Legal Information", and "Privacy Statement", along with the SAS logo and copyright information: "Copyright © 2026 SAS Institute Inc. All Rights Reserved."

SAS® OnDemand for Academics

United States 1

Man Tsui

SAS® Studio

Write and run SAS code with a Web-based SAS development environment.

[Launch](#)

Other Actions:

- [Learn](#) to use this client.
- [Clear my saved tabs](#)

SAS Training

Master analytics skills—start learning free with expert-led SAS training.

[Learn More](#)

[License](#) | [Terms of Use & Legal Information](#) | [Privacy Statement](#)

sas

Copyright © 2026 SAS Institute Inc. All Rights Reserved.

A SAS PROGRAM

Let's read the following into SAS to generate basic descriptive statistics:

SUBJECT NUMBER	GENDER (M or F)	EXAM 1	EXAM 2	HOMEWORK GRADE
10	M	80	84	A
7	M	85	89	A
4	F	90	86	B
20	M	82	85	B
14	F	88	NA	C

A SAS PROGRAM

```
data grades;
  INFILE datalines;
  INPUT SUBJECT 1-2 GENDER $ 4 EXAM1 6-8 EXAM2 10-12
         HWGRADE $ 14;
  DATALINES;
10 M 80 84 A
 7 M 85 89 A
 4 F 90 86 B
20 M 82 85 B
14 F 88 . C
;
RUN;
PROC MEANS DATA=grades;
RUN;
```

A SAS PROGRAM

```
data grades;
  infile datalines;
  input subject 1-2 gender $ 4 exam1 6-8 exam2 10-12
        hwgrade $ 14;
datalines;
10 M 80 84 A
 7 M 85 89 A
 4 F 90 86 B
20 M 82 85 B
14 F 88 . C
;
run;
proc means data=grades;
run;
```

Lowercase works.

A SAS PROGRAM

```
/* DATA step reads & manipulates data */
data grades;
  infile datalines;
  input subject 1-2 gender $ 4 exam1 6-8 exam2 10-12
        hwgrade $ 14;
  datalines;
10 M   80   84 A
 7 M   85   89 A
 4 F   90   86 B
20 M   82   85 B
14 F   88   .  C
;
run;
/* PROCEDURE step analyzes data */
proc means data=test;
run;
```

You can have as many DATA steps (read-in data) and PROC steps (mostly to analyze data) as you want.

A SAS PROGRAM

```
/* DATA step reads & manipulates data */  
data grades;  
    infile datalines;  
    input subject 1-2 gender $ 4 exam1 6-8 exam2 10-12  
        hwgrade $ 14;  
datalines;  
10 M    80    84 A  
 7 M    85    89 A  
 4 F    90    86 B  
20 M    82    85 B  
14 F    88    . C  
;  
run;  
/* PROCEDURE step analyzes data */  
proc means data=test;  
run;
```

- We first show what you see in SAS Studio.
- Then we show some PROC steps you can use.
- Lastly, we deal with DATA step.

SAS STUDIO: WHERE YOUR "GRADES" IS STORED: OUTPUT DATA

SAS® Studio

Server Files and Folders

Tasks and Utilities

Snippets

Libraries

- My Libraries
 - MAPS
 - MAPSGFK
 - MAPSSAS
 - SASHELP
 - SASUSER
 - WEBWORK
 - WORK
 - GRADES

*Program 1.sas

CODE LOG RESULTS **OUTPUT DATA**

Table: WORK.GRADES View: Column names Filter: (none)

Total rows: 5 Total columns: 5

	subject	gender	exam1	exam2
1	10	M	80	84
2	7	M	85	89
3	4	F	90	86
4	20	M	82	85
5	14	F	88	.

Columns

- Select all
- subject
- gender
- exam1
- exam2
- hwgrade

Property Value

Label	
Name	
Length	
Type	
Format	

File Shortcuts

SAS STUDIO: WHERE YOUR "GRADES" IS STORED: LIBRARIES > WORK.GRADES

SAS® Studio

Server Files and Folders

Tasks and Utilities

Snippets

Libraries

- My Libraries
 - MAPS
 - MAPSGFK
 - MAPSSAS
 - SASHELP
 - SASUSER
 - WEBWORK
 - WORK
 - GRADES

*Program 1.sas

CODE LOG RESULTS OUTPUT DATA

Table: WORK.GRADES View: Column names Filter: (none)

Columns

- Select all
- subject
- gender
- exam1
- exam2
- hwgrade

Property Value

subject	gender	exam1	exam2	hwgrade
10	M	80	84	
7	M	85	89	
4	F	90	86	
20	M	82	85	
14	F	88		

PROC STEP

Some example procedures

PROC MEANS

```
proc means data=grades;  
run;
```

The MEANS Procedure

Variable	N	Mean	Std Dev	Minimum	Maximum
subject	5	11.0000000	6.2449980	4.0000000	20.0000000
exam1	5	85.0000000	4.1231056	80.0000000	90.0000000
exam2	4	86.0000000	2.1602469	84.0000000	89.0000000

PROC MEANS WITH FEWER VARIABLES DISPLAYED

```
proc means data=grades;  
    var exam1 exam2;  
run;
```

The MEANS Procedure

Variable	N	Mean	Std Dev	Minimum	Maximum
exam1	5	85.0000000	4.1231056	80.0000000	90.0000000
exam2	4	86.0000000	2.1602469	84.0000000	89.0000000

PROC MEANS WITH FEWER VARIABLES AND FEWER STATISTICS

```
proc means data=grades mean stdev;  
    var exam1 exam2;  
run;
```

The MEANS Procedure

Variable	Mean	Std Dev
exam1	85.0000000	4.1231056
exam2	86.0000000	2.1602469

PROC UNIVARIATE

```
proc means data=grades;  
    var exam1 exam2;  
run;
```

Returns moments, statistical measures, statistical tests, confidence intervals, quantiles, and more.

PROC SORT

```
proc sort data=grades;  
  by exam1;  
run;
```

Sorts in place (i.e. changes original copy) unless you add option `out=new_file_name`

Obs	subject	gender	exam1	exam2	hwgrade
1	10	M	80	84	A
2	20	M	82	85	B
3	7	M	85	89	A
4	14	F	88	.	C
5	4	F	90	86	B

PROC SORT WITH OUT OPTION

```
proc sort data=grades out=newgrades;  
    by exam1;  
run;
```

PROC PRINT

```
proc print data=grades;  
  title "Grades";  
  var exam1 exam2;  
run;
```

Grades		
Obs	exam1	exam2
1	80	84
2	85	89
3	90	86
4	82	85
5	88	.

DATA STEP

Reading in data

DATA STEP

```
/* DATA step reads & manipulates data */
data grades;
    infile datalines;
    input subject 1-2 gender $ 4 exam1 6-8 exam2 10-12
           hwgrade $ 14;
datalines;
10 M    80    84 A
 7 M    85    89 A
 4 F    90    86 B
20 M    82    85 B
14 F    88     . C
;
run;
/* PROCEDURE step analyzes data */
proc means data=test;
run;
```

Add newlines to the `input` statement to more clearly see what is going on...

DATA STEP

```
/* DATA step reads & manipulates data */
data grades; /* dataset stored as variable "grades" */
  infile datalines;
  input
    subject 1-2 /* Numeric var. "subject" uses col. 1-2 */
    gender $ 4 /* Char. var. "gender" uses col. 4 */
    exam1 6-8
    exam2 10-12
    hwgrade $ 14;
datalines;
10 M 80 84 A
 7 M 85 89 A
 4 F 90 86 B
20 M 82 85 B
14 F 88 . C
;
run;
```

"\$" specifies character variable

DATA STEP

```
/* DATA step reads & manipulates data */
data grades; /* dataset stored as variable "grades" */
  infile datalines;
  input
    subject 1-2 /* Numeric var. "subject" uses col. 1-2 */
    gender $ 4 /* Char. var. "gender" uses col. 4 */
    exam1 6-8
    exam2 10-12
    hwgrade $ 14;
  datalines;
10 M 80 84 A
 7 M 85 89 A
 4 F 90 86 B
20 M 82 85 B
14 F 88 . C
;
run;
```

"exam1 6-8" means exam1 data are in columns 6-8
(when read from the row or punchcard).

DATA STEP

```
/* DATA step reads & manipulates data */
data grades; /* dataset stored as variable "grades" */
  infile datalines;
  input
    subject 1-2 /* Numeric var. "subject" uses col. 1-2 */
    gender $ 4 /* Char. var. "gender" uses col. 4 */
    exam1 6-8
    exam2 10-12
    hwgrade $ 14;
  datalines;
10 M    80    84 A
  7 M    85    89 A
  4 F    90    86 B
20 M    82    85 B
14 F    88     . C
;
run;
```

In DATALINES: Numeric data are right-aligned
(convention)

DATA STEP

```
/* DATA step reads & manipulates data */
data grades; /* dataset stored as variable "grades" */
  infile datalines;
  input
    subject 1-2 /* Numeric var. "subject" uses col. 1-2 */
    gender $ 4 /* Char. var. "gender" uses col. 4 */
    exam1 6-8
    exam2 10-12
    hwgrade $ 14;
datalines;
10 M 80 84 A
 7 M 85 89 A
 4 F 90 86 B
20 M 82 85 B
14 F 88 . C
;
run;
```

In DATALINES: Missing data is written as .

DATA STEP: COLUMN INPUT (LIKE PUNCH CARDS)

```
/* DATA step reads & manipulates data */
data grades; /* dataset stored as variable "grades" */
  infile datalines;
  input
    subject 1-2 /* Numeric var. "subject" uses col. 1-2 */
    gender $ 4 /* Char. var. "gender" uses col. 4 */
    exam1 6-8
    exam2 10-12
    hwgrade $ 14;
datalines;
10 M 80 84 A
 7 M 85 89 A
 4 F 90 86 B
20 M 82 85 B
14 F 88 . C
;
run;
```

State columns used in column input.

```

/* DATA step reads & manipulates data */
data grades; /* dataset stored as variable "grades" */
  infile datalines;
  input
    subject
    gender $
    exam1
    exam2
    hwgrade $;
datalines;
10 M 80 84 A
 7 M 85 89 A
 4 F 90 86 B
20 M 82 85 B
14 F 88 . C
;
run;

```

List input is space-separated (even if columns aren't aligned). Each variable is given 8 bytes by default so strings are 8-digits long.

```

/* DATA step reads & manipulates data */
data grades; /* dataset stored as variable "grades" */
  infile datalines dsd;
  input
    subject
    gender $
    exam1
    exam2
    hwgrade $;
  datalines;
10, M, 80, 84, A
 7, M, 85, 89, A
 4, F, 90, 86, B
20, M, 82, 85, B
14, F, 88, ., C
;
run;

```

DSD = delimiter sensitive data, with default delimiter comma. When two commas are hit, this option knows a missing value is in between.

DATA STEP: LIST INPUT WITH TAB DELIMITER

```
/* DATA step reads & manipulates data */
data grades; /* dataset stored as variable "grades" */
  infile datalines dlm = '09'x dsd;
  input
    subject
    gender $
    exam1
    exam2
    hwgrade $;
datalines;
10      M      80      84      A
7       M      85      89      A
4       F      90      86      B
20      M      82      85      B
14      F      88      .      C
;
run;
```

READING IN SPECIAL DATA WITH INFORMAT

	Example	INFORMAT
Date	04/11/1982 OR 11-12-1962 OR 11.12.1962	mmddyy10.
	11121962 OR 11/12/62	mmddyy8
	12NOV1962	date9
Comma in number	59,365	comma6.
Long (>8) characters	086-59-9054	\$11.
Long decimal number	12345.600012	5.6

See <https://www.listendata.com/2023/07/sas-formats.html> for
more

READING SPECIAL DATA WITH INFORMAT: CORRECT INFORMAT BUT WRONG OUT FORMAT

```
DATA Hospital;  
INFILE Datalines DSD;  
INFORMAT dob admit dischrg mmddyy10.;  
INPUT id$ dob admit dischrg fee;  
DATALINES;  
001,10/21/1946,12/12/2004,12/14/2004,8000  
002,05/01/1980,07/08/2004,08/08/2004,12000  
003,01/01/1960,01/01/2004,01/04/2004,9000  
004,06/23/1998,11/11/2004,12/25/2004,15123  
;  
PROC PRINT DATA=Hospital;  
RUN;
```

Obs	dob	admit	dischrg	id	fee
1	-4820	16417	16419	001	8000
2	7426	16260	16291	002	12000
3	0	16071	16074	003	9000
4	14053	16386	16430	004	15123

Shows days since January 1, 1960

READING SPECIAL DATA WITH INFORMAT: CORRECT INFORMAT & OUT FORMAT

```
DATA Hospital;
INFILE Datalines DSD;
INFORMAT dob admit dischrg mmddyy10.;
INPUT id$ dob admit dischrg fee;
DATALINES;
001,10/21/1946,12/12/2004,12/14/2004,8000
002,05/01/1980,07/08/2004,08/08/2004,12000
003,01/01/1960,01/01/2004,01/04/2004,9000
004,06/23/1998,11/11/2004,12/25/2004,15123
;
PROC PRINT DATA=Hospital;
FORMAT dob admit dischrg mmddyy10.;
RUN;
```

Total rows: 4 Total columns: 5

Rows 1-4

	dob	admit	dischrg	id	fee
1	-4820	16417	16419	001	8000
2	7426	16260	16291	002	12000
3	0	16071	16074	003	9000
4	14053	16386	16430	004	15123

READING SPECIAL DATA WITHOUT INFORMAT: USE COLON MODIFIER

```
DATA info;  
  INFILE DATALINES;  
  INPUT ssn      : $11.  
         taxdate : mmddyy10.  
         income  : comma6. ;  
  DATALINES;  
086-59-9054 04/12/2001 59,365  
405-65-0987 03/15/2002 26,925  
212-44-9054 04/15/2003 44,999  
;
```

INFORMAT can also be specified within INPUT if you use colon modifier.

IMPORT FILES: CSV FILE

Use PROC IMPORT if the data file is clean (rectangular, CSV, etc.)

```
PROC IMPORT
    DATAFILE='billboards.csv' /* location */
    OUT = billboards           /* name */
    DBMS = csv                 /* file type */
    REPLACE ;                  /* Override old dataset, if exi
    GETNAMES = yes;            /* Row 1 = variables names? */
RUN;

PROC PRINT DATA= billboards (OBS=5);
PROC CONTENTS DATA= billboards;
RUN;
```

IMPORT FILES: EXCEL (XLSX) FILE

Use PROC IMPORT if the data file is clean (rectangular, CSV, etc.)

```
PROC IMPORT
    DATAFILE='billboards.xlsx' /* location */
    OUT = billboards             /* name */
    DBMS = xlsx                  /* file type */
    REPLACE ;                    /* Override old dataset, if exist
    GETNAMES = yes;              /* Row 1 = variables names? */
RUN;

PROC PRINT DATA= billboards (OBS=5);
PROC CONTENTS DATA= billboards;
RUN;
```

IMPORT FILES: FROM THE INTERNET

```
filename myurl url "https://manctsui.github.io/datasets/billbo
PROC IMPORT
    DATAFILE=myurl
    OUT = billboards
    DBMS = csv
    REPLACE ;
    GETNAMES = yes;
RUN;
```

The "Import Data" option imports data by generating a PROC IMPORT.

The screenshot displays the SAS Studio interface. On the left, the 'Server Files and Folders' pane shows a context menu for 'Program 1.sas' with options: 'SAS Program (F4)', 'Import Data', 'Query', 'XML', 'Folder', and 'Folder Shortcut'. The 'Import Data' option is highlighted, and a sub-menu item 'New Import Data' is visible. The main editor window shows 'Program 1.sas' with the following SAS code:

```
1 data test; /* dataset stored as variable "test" */
2   input
3     subject /* Numeric var. "subject" uses col. 1-2 */
4     gender $ /* Char. var. "gender" uses col. 4 */
5     exam1
6     exam2
7     hwgrade $ ;
8 datalines;
9 10 M 80 84 A
10 7 M 85 89 A
11 4 F 90 86 B
12 20 M 82 85 B
13 12 F 88 . C
14 ;
15 run;
```

The bottom status bar indicates the file path: `/home/u64546656/sasuser.v94/Program 1.sas`, the cursor position: `Line 1, Column 10`, the encoding: `UTF-8`, and the user: `User: u64546656`.

DATA STORED IN WORK FOLDER

WORK folder is temporary, so store permanent output in Server Files and Folders.

The screenshot displays the SAS Studio web interface. On the left, the 'Libraries' pane shows the 'WORK' folder selected under 'My Libraries'. The main editor window, titled 'Program 1.sas', shows a SAS program in the 'CODE' tab. The program creates a dataset named 'test' and contains data for 12 subjects. The status bar at the bottom indicates the file path as '/home/u64546656/sasuser.v94/Program 1.sas' and the cursor position as 'Line 1, Column 10' in 'UTF-8' encoding.

```
1 data test; /* dataset stored as variable "test" */
2   input
3     subject /* Numeric var. "subject" uses col. 1-2 */
4     gender $ /* Char. var. "gender" uses col. 4 */
5     exam1
6     exam2
7     hwgrade $ ;
8 datalines;
9 10 M 80 84 A
10 7 M 85 89 A
11 4 F 90 86 B
12 20 M 82 85 B
13 12 F 88 . C
14 ;
15 run;
```

EXPORT DATA

SAS® Studio

Server Files and Folders

Tasks and Utilities

Snippets

Libraries



My Libraries

MAPS

MAPSGFK

MAPSSAS

SASHELP

SASUSER

WEBWORK

WORK

HOSPITAL

TEST

New Library

Delete

Rename

Duplicate

Export

Properties

*Program 1.sas

CODE

LOG

RESULTS

OUTPUT DATA



```
1 DATA Hospital;
2 INFILE Datalines DSD;
3 INFORMAT dob admit dischrg mmddyy10.;
4 INPUT id$ dob admit dischrg fee;
5 DATALINES;
6 001,10/21/1946,12/12/2004,12/14/2004,8000
7 002,05/01/1980,07/08/2004,08/08/2004,12000
8 003,01/01/1960,01/01/2004,01/04/2004,9000
9 004,06/23/1998,11/11/2004,12/25/2004,15123
10 ;
11 PROC PRINT DATA=info;
12 FORMAT taxdate mmddyy10. income comma6.;
13 RUN;
14
15
```

MORE DATA STEP OPTIONS

```
data grades;
  infile datalines;
  input subject 1-2 gender $ 4 exam1 6-8 exam2 10-12
        hwgrade $ 14;
  /* ----- */
  /* Do more here! */
  /* ----- */
datalines;
10 M 80 84 A
 7 M 85 89 A
 4 F 90 86 B
20 M 82 85 B
14 F 88 . C
;
run;
```

We can do more in DATA step between INPUT and
DATALINES

MORE DATA STEP OPTIONS: KEEP/DROP

```
data grades;
  infile datalines;
  input subject 1-2 gender $ 4 exam1 6-8 exam2 10-12
        hwgrade $ 14;
  keep subject gender; /* Keep certain columns */
datalines;
10 M 80 84 A
7 M 85 89 A
4 F 90 86 B
20 M 82 85 B
14 F 88 . C
;
run;
```

	subject	gender
1	10	M
2	7	M
3	4	F
4	20	M
5	14	F

COLUMNS

```
data grades;
  infile datalines;
  input subject 1-2 gender $ 4 exam1 6-8 exam2 10-12
        hwgrade $ 14;
  examavg = (exam1+exam2)/2; /* Create column "examavg" */
datalines;
10 M 80 84 A
 7 M 85 89 A
 4 F 90 86 B
20 M 82 85 B
14 F 88 . C
;
run;
```

	subject ^	gender	exam1	exam2	hwgrade	examavg
1	4	F	90	86	B	88
2	7	M	85	89	A	87
3	10	M	80	84	A	82
4	14	F	88	.	C	.
5	20	M	82	85	B	82.5

5		20	M		82		85	B		83.5
---	--	----	---	--	----	--	----	---	--	------

REFERENCES

- <https://its.uri.edu/sas-tutorial-2/> for types of inputs
- We will partly follow the previous iteration of this course, as well as *Applied Statistics and the SAS Programming Language* 4th and 5th editions, and other textbooks.